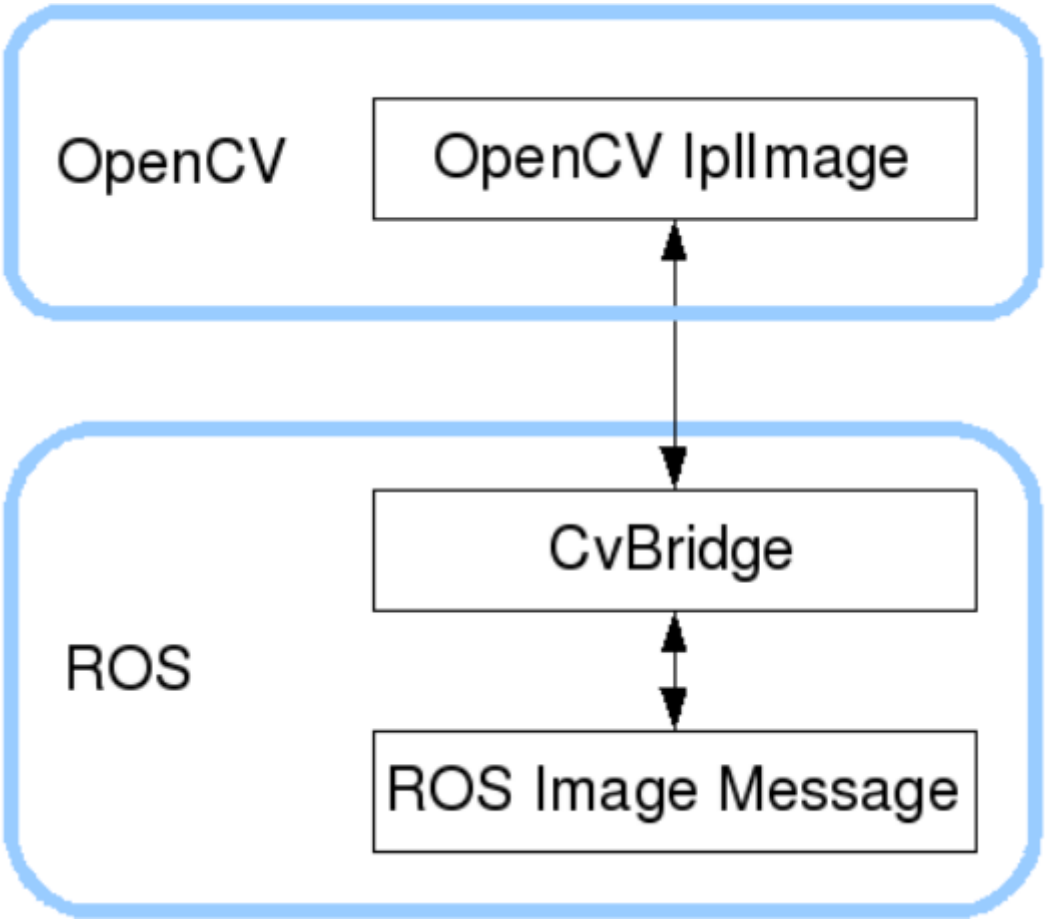
ROS + OpenCV Applications

ROS transmits images using its own proprietary `sensor_msgs/Image` message format, which cannot be directly processed by standard image processing libraries. However, the provided **CvBridge** tool enables seamless conversion of image data formats. **CvBridge** is a ROS library that serves as a bridge between ROS and OpenCV.

The process for converting image data between OpenCV and ROS is illustrated in the diagram below:



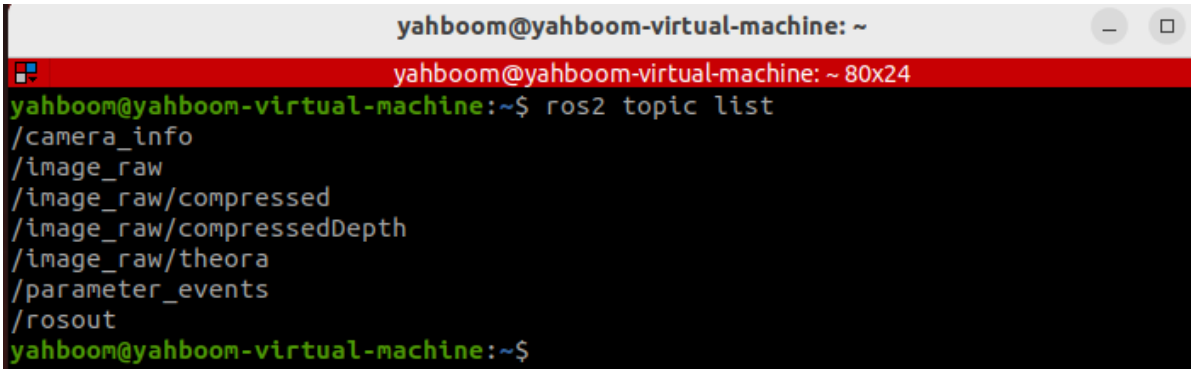
Launch the inverse kinematics program and the camera program:

```
ros2 launch dofbot_info  camera_arm_kin.launch.py
```

1.1.2. Viewing Camera Topics

Enter the following command in the terminal:

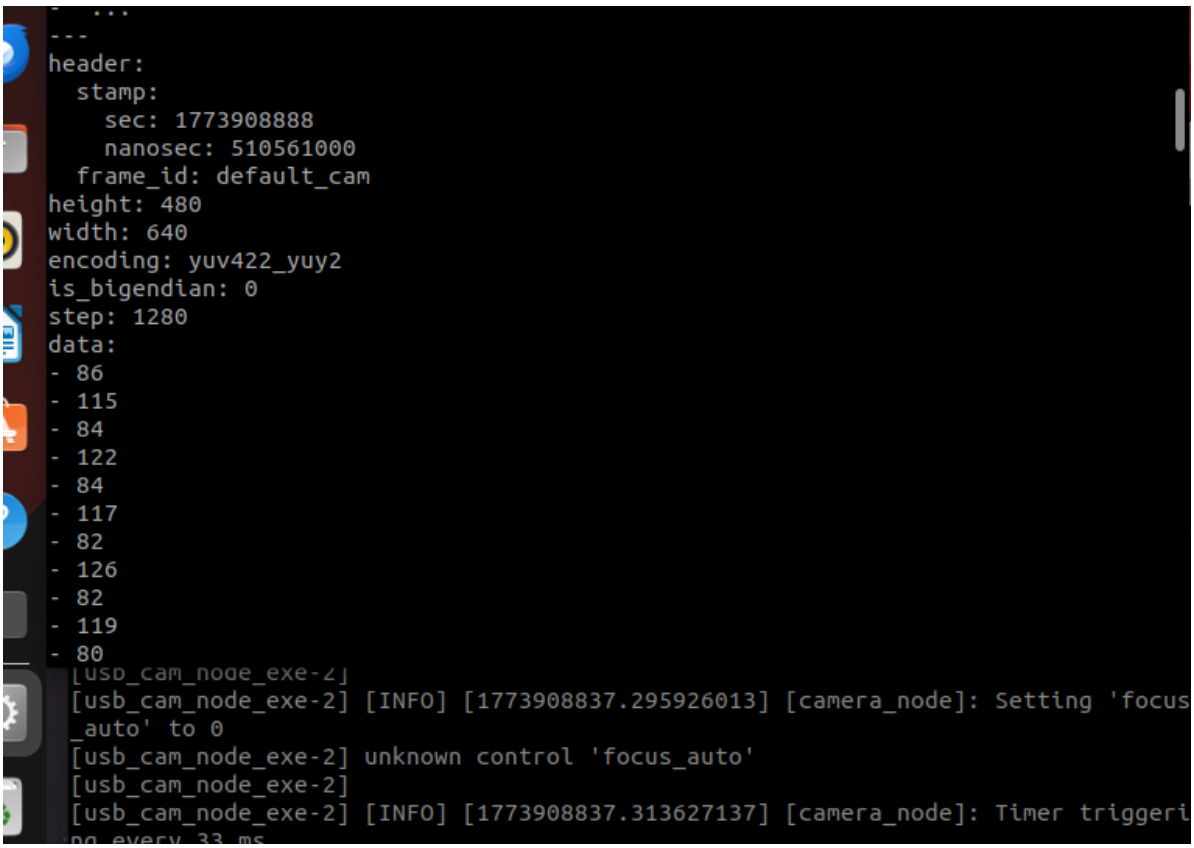
```
ros2 topic list
```



The primary focus here is on the image data topics; specifically, we will analyze the information regarding the RGB color image topic. Use the following command to inspect the data content of this topic by entering it into the terminal:

```
# View the data content of the RGB image topic
ros2 topic echo /image_raw
```

First, let's capture a single frame of the RGB color image data:



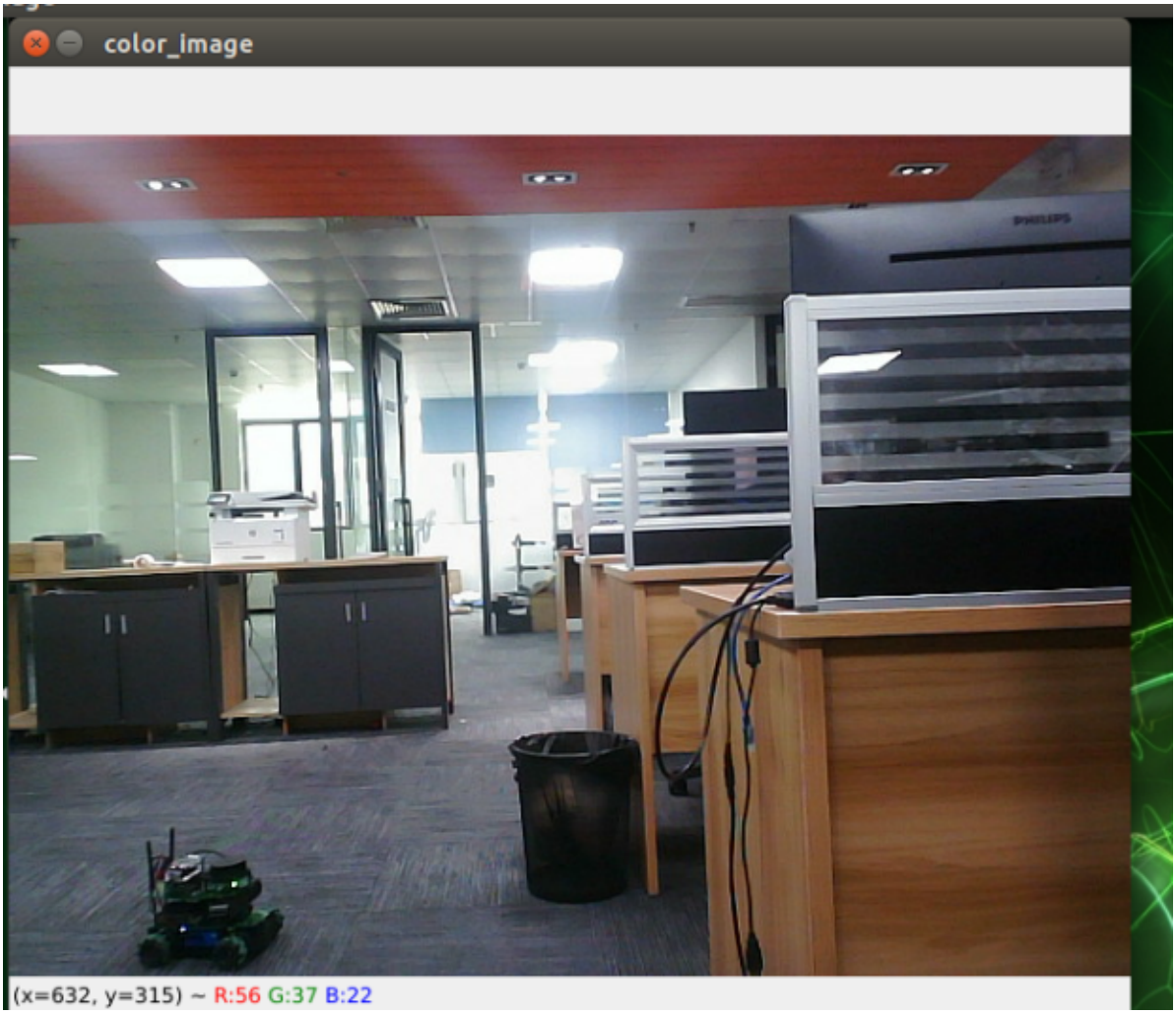
This output displays the basic information regarding the image. One particularly important value is **encoding**; in this instance, the value is **yuv422_yuy2**. This value indicates that the image data is being stored and transmitted using the YUV422 format with YUY2 encoding.

2. Subscribing to RGB Image Topics and Displaying the RGB Image

2.1. Running the Command

Enter the following in the terminal:

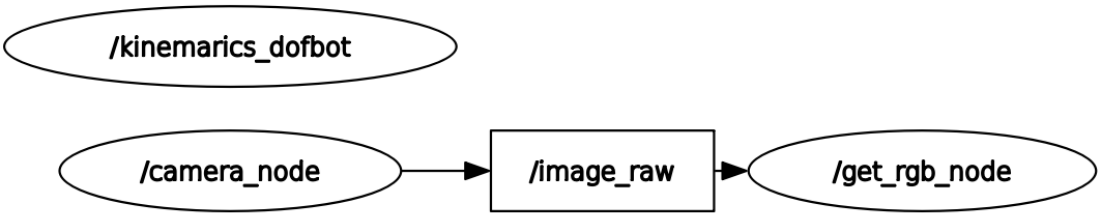
```
# RGB Color Image Display Node
ros2 run yahboom_vision rgb_image
```



2.2. Viewing the Node Communication Graph

Enter the following in the terminal:

```
ros2 run rqt_graph rqt_graph
```



2.3. Core Code Analysis

Code Reference Path:

```
/home/yahboom/dofbot_ws/src/yahboom_vision/yahboom_vision/rgb_image.py
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2


class AstraRGBImage(Node):
    """
    ROS 2 node that subscribes to RGB images, resizes them, and republishes.
    ROS 2 节点：订阅 RGB 图像，调整尺寸，然后重新发布。
    """

    def __init__(self, name):
        """
        Constructor - Initialize the node with subscriptions and publishers.
        构造函数 - 初始化节点，创建订阅者和发布者。

        Args:
            name (str): The name of the ROS node / ROS 节点的名称
        """
        super().__init__(name)

        # Create CvBridge for converting between ROS Image messages and OpenCV
images
        # 创建 CvBridge，用于在 ROS Image 消息和 OpenCV 图像之间转换
        self.bridge = CvBridge()

        # Subscribe to the raw image topic from camera or other source
        # 订阅来自摄像头或其他源的原始图像话题
        self.sub_img = self.create_subscription(
            Image,                          # Message type / 消息类型
            '/image_raw',                   # Topic name / 话题名称
            self.handleTopic,               # Callback function / 回调函数
            100                             # Queue size / 队列大小
        )

        # Publisher for processed/resized images
        # 发布处理后的（已调整尺寸的）图像的发布者
        self.pub_img = self.create_publisher(
            Image,                          # Message type / 消息类型
            '/processed_image',             # Output topic name / 输出话题名称
            100                             # Queue size / 队列大小
        )


    def handleTopic(self, msg):
        """
        Callback function - Process incoming image messages.
        回调函数 - 处理接收到的图像消息。
```

```
        """
        # Check if received message is of correct type
        # 检查接收到的消息是否为正确的类型
        if not isinstance(msg, Image):
            self.get_logger().error('Received non-Image message')
            return

        try:
            # Convert ROS Image message to OpenCV BGR format image
            # 将 ROS Image 消息转换为 OpenCV BGR 格式图像
            frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

            # Resize image to standardized dimensions (640x480 pixels)
            # 将图像调整到标准尺寸（640×480 像素）
            frame = cv2.resize(frame, (640, 480))

            # Convert processed OpenCV image back to ROS Image message
            # 将处理后的 OpenCV 图像转换回 ROS Image 消息
            output_msg = self.bridge.cv2_to_imgmsg(frame, 'bgr8')

            # Copy header information (timestamp, frame_id) from original
message
            # 复制原始消息的头信息（时间戳、坐标系 ID）
            output_msg.header = msg.header

            # Publish the processed image to the output topic
            # 将处理后的图像发布到输出话题
            self.pub_img.publish(output_msg)

            # Display the image in an OpenCV window
            # 在 OpenCV 窗口中显示图像
            cv2.imshow('color_image', frame)
            cv2.waitKey(10)

        except Exception as e:
            self.get_logger().error(f'Error processing image: {str(e)}')
```